



Experiment 3

Student Name: Mohammad Taha

UID:24BCS12733

Branch:BE-CSE

Section/Group:719-‘A’

Semester:5th

Date:01/09/2026

Subject Name: PBLJ

Subject Code: 24CSH-301

Aim: Design Java programs showcasing exception handling through square root calculations, an ATM withdrawal system, and a university enrollment system with custom exceptions.

EASY LEVEL

1. Problem Statement: To implement a Java program to calculate the square root of a number entered by the user and demonstrate exception handling using try-catch-finally for invalid and negative inputs.

2. Objective:

- To understand the basic concept of exception handling in Java.
- To use `try-catch` blocks to handle runtime errors.
- To handle negative numbers using a custom-thrown exception.
- To use the `finally` block for resource management.

3. Code:

```
import java.util.Scanner;

public class SquareRootCalculator {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        try {
            System.out.print("Enter a number: ");
            double number = scanner.nextDouble();
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

```
        if (number < 0) {  
            throw new IllegalArgumentException(  
                "Error: Cannot calculate the square root of a negative number."  
            );  
        }  
  
        double result = Math.sqrt(number);  
  
        System.out.println("Square root: " + result);  
  
    } catch (IllegalArgumentException e) {  
        System.out.println(e.getMessage());  
    }  
  
    } catch (Exception e) {  
        System.out.println(  
            "Error: Invalid input. Please enter a valid number."  
        );  
    }  
  
    } finally {  
        scanner.close();  
    }  
    }  
}
```

4. Output:

```
Taha@LAPTOP-SAKCHV1L MINGW64 ~/Desktop/java sem5/exp3  
$ /usr/bin/env C:\Program Files\Java\jdk-26\bin\java.exe --enable-preview -  
\Code\User\workspaceStorage\95e0dc50e76b81cb3d545a15b4ac9477\redhat.java\jdt_  
● Enter a number: 67  
Square root: 8.18535277187245
```

Problem -2

1. Problem Statement: - To implement a Java ATM withdrawal system using exception handling for invalid PIN and insufficient account balance, while ensuring that the remaining balance is displayed using the finally block.

2. Objective:

- To understand the use of custom exceptions in Java.
- To validate the PIN entered by the user.
- To handle insufficient account balance using an exception.
- To use multiple `catch` blocks for different exceptions.
- To demonstrate the use of `finally` for displaying the current balance.

3. Code:

```
import java.util.Scanner;

class InvalidPinException extends Exception {

    public InvalidPinException(String message) {
        super(message);
    }
}

class InsufficientBalanceException extends Exception {

    public InsufficientBalanceException(String message) {
        super(message);
    }
}

public class ATMWithdrawalSystem {

    private static final int CORRECT_PIN = 1234;
    private static double balance = 3000.0;

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

```
try {
    System.out.print("Enter PIN: ");
    int enteredPin = scanner.nextInt();

    if (enteredPin != CORRECT_PIN) {
        throw new InvalidPinException(
            "Error: Invalid PIN. Please try again."
        );
    }

    System.out.print("Withdraw Amount: ");
    double withdrawAmount = scanner.nextDouble();

    if (withdrawAmount <= 0) {
        throw new IllegalArgumentException(
            "Error: Withdrawal amount must be greater than zero."
        );
    }

    if (withdrawAmount > balance) {
        throw new InsufficientBalanceException(
            "Error: Insufficient balance."
        );
    }

    balance -= withdrawAmount;

    System.out.println(
        "Withdrawal Successful!"
    );

} catch (InvalidPinException | InsufficientBalanceException e) {
    System.out.println(e.getMessage());
} catch (IllegalArgumentException e) {
    System.out.println(e.getMessage());
} catch (Exception e) {
    System.out.println(
        "Error: Invalid input. Please enter numeric values."
    );
} finally {
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

```
        System.out.println("Current Balance: " + balance);
        scanner.close();
    }
}
```

4. Output:

```
Taha@LAPTOP-SAKCHV1L MINGW64 ~/Desktop/java sem5/exp3
$ /usr/bin/env C:\\Program\\ Files\\Java\\jdk-26\\bin\\java.exe --enable-
AppData\\Roaming\\Code\\User\\workspaceStorage\\95e0dc50e76b81cb3d545a15b
Enter PIN: 7783
Error: Invalid PIN. Please try again.
Current Balance: 3000.0

Taha@LAPTOP-SAKCHV1L MINGW64 ~/Desktop/java sem5/exp3
$ /usr/bin/env C:\\Program\\ Files\\Java\\jdk-26\\bin\\java.exe --enable-
AppData\\Roaming\\Code\\User\\workspaceStorage\\95e0dc50e76b81cb3d545a15b
Enter PIN: 1234
Withdraw Amount: 600
Withdrawal Successful!
Current Balance: 2400.0
```

Hard Level

1. Problem Statement: To design and implement a Java university enrollment system using custom exceptions to handle full courses and unmet prerequisites during student enrollment.

2. Objective:

- To understand the use of custom exception classes in Java.
- To implement exception handling for course enrollment.
- To handle courses that have reached their maximum capacity.
- To verify prerequisite requirements before enrollment.
- To use `throw`, `throws`, `try`, `catch`, and `finally` in a practical application.

3. Code:

```
import java.util.*;

class CourseFullException extends Exception {

    public CourseFullException(String message) {
        super(message);
    }
}

class PrerequisiteNotMetException extends Exception {

    public PrerequisiteNotMetException(String message) {
        super(message);
    }
}

class Course {

    private String name;
    private int maxCapacity;
    private int enrolledStudents;
    private String prerequisite;

    public Course(String name, int maxCapacity, String prerequisite) {
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

```
this.name = name;
this.maxCapacity = maxCapacity;
this.enrolledStudents = 0;
this.prerequisite = prerequisite;
}

public void enrollStudent(Set<String> completedCourses)
    throws CourseFullException, PrerequisiteNotMetException {

    if (enrolledStudents >= maxCapacity) {
        throw new CourseFullException(
            "Error: CourseFullException - " +
            name + " has reached maximum capacity."
        );
    }

    if (prerequisite != null &&
        !prerequisite.isEmpty() &&
        !completedCourses.contains(prerequisite)) {

        throw new PrerequisiteNotMetException(
            "Error: PrerequisiteNotMetException - Complete " +
            prerequisite + " before enrolling in " + name + "."
        );
    }

    enrolledStudents++;

    System.out.println(
        "Successfully enrolled in " + name +
        ". Remaining slots: " +
        (maxCapacity - enrolledStudents)
    );
}

}

public class UniversityEnrollmentSystem {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        Course coreJava = new Course("Core Java", 2, "");
        Course advancedJava = new Course("Advanced Java", 2, "Core Java");
```



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

```
Set<String> completedCourses = new HashSet<>();

try {
    System.out.print(
        "Have you completed Core Java? (yes/no): "
    );

    String input = scanner.nextLine().trim().toLowerCase();

    if (input.equals("yes")) {
        completedCourses.add("Core Java");
    }

    System.out.println("\nEnrolling in Advanced Java...");

    advancedJava.enrollStudent(completedCourses);

    System.out.println(
        "Enrollment process completed successfully."
    );

} catch (CourseFullException |
        PrerequisiteNotMetException e) {

    System.out.println(e.getMessage());

} catch (Exception e) {

    System.out.println("Error: Invalid input.");

} finally {
    scanner.close();
}
}
```




4. Output:

```
Taha@LAPTOP-SAKCHV1L MINGW64 ~/Desktop/java sem5/exp3
$ /usr/bin/env C:\Program Files\Java\jdk-26\bin\java.exe --enable-preview -XX
AppData\Roaming\Code\User\workspaceStorage\95e0dc50e76b81cb3d545a15b4ac9477\re
System
● Have you completed Core Java? (yes/no): yes

Enrolling in Advanced Java...
Successfully enrolled in Advanced Java. Remaining slots: 1
Enrollment process completed successfully.
```

5. Learning Outcome:

- Understand the fundamental concept of exception handling in Java.
- Use `try`, `catch`, and `finally` blocks to handle runtime errors.
- Identify and handle invalid user input.
- Use the `throw` keyword to generate exceptions explicitly.
- Create and use custom exception classes.
- Handle multiple exceptions using appropriate `catch` blocks.
- Use the `throws` keyword for exception propagation.
- Apply exception handling to practical systems such as calculators, ATMs, and university enrollment.
- Validate conditions and prevent invalid operations through exception handling.
- Develop more reliable and fault-tolerant Java applications.